

SURF-2026-0717-Summary-Chinese-20260804

问题概述

在传统停车问题（Parking Function）中，考虑 n 辆车停入 n 个车位，每辆车都有一个偏好车位，车会尽量停到偏好车位。

例如，在 $n = 3$ 的Parking Function中（即考虑3辆车和3个车位）。用向量 $\vec{s} = (0, 0, 0)$ 表示一条有三个车位的路，车辆从向量左侧驶入，其中 0 表示车位是空的。用向量 $\vec{p} = (3, 1, 2)$ 表示三辆车的偏好车位：第一辆车（用 C_1 表示）偏好第三个位置（用 s_3 表示， s_3 表示 $\vec{s}$ 的第3项），因为 $p_1 = 3$ 。同理， C_2 偏好 s_1 ， C_3 偏好 s_2 。那么最终这条路的停车情况可以用 $\vec{s} = (2, 3, 1)$ 表示， C_1 停在 s_3 ， C_2 停在 s_1 ， C_3 停在 s_2 ，即所有车都如愿停车。

停车顺序和车的序号一致，即 C_1 第一个停车， C_2 第二个停车，以此类推。

但是，如果有的车在停车时发现自己的偏好车位被占，例如，在 $n = 4$ 的CSPF中， $\vec{s} = (0, 1, 2, 0)$ ，现在 C_3 准备停车，而且 $p_3 = 3$ ，而 $s_3 = 2$ ，所以 C_3 需要找到其他车位停车。可以看到， $s_1 = s_4 = 0$ ，这两个车位是 C_3 的备选。在Closest-Spot Parking Function（CSPF）中规定：如果某辆车发现自己的偏好车位被占，它就会停在离这个偏好车位最近的空车位。所以 C_3 最终停在 s_4 ，因为 s_4 到 C_3 的偏好位置 s_3 的距离比 s_1 更近（规定两个相邻车位的距离为1）。

同理，如果 $\vec{s} = (0, 1, 2, 0)$ 且 $p_3 = 2$ ，那么在 C_3 会停在 s_1 。

CSPF规定，如果某辆车的偏好位置已经被占，有两个空车位到这个偏好车位的距离相等，而且没有比这两个车空位更近的空车位，那么这辆车会选择左侧，也就是序号更小的那个车位。例如，在 $n = 3$ 的车位中， $\vec{s} = (0, 1, 0)$ ， $p_2 = 2$ ，此时 C_2 需要停在 s_1 或 s_3 ，它们到 s_2 的距离相等，根据规定， C_2 会把车停在 s_1 ，因为它的序号更小。

定义与问题

这部分介绍关于CSPF的一些定义和围绕CSPF展开的一些问题。

相关定义：

- 定义向量 $\vec{r}$ 表示每辆车最终停车的位置，比如当 $\vec{s} = (2, 3, 1)$ 时， $\vec{r} = (3, 1, 2)$ ， $r_1 = 3$ 表示 C_1 最终停在 s_3 ， $r_2 = 1$ 表示 C_2 最终停在 s_1 ，依此类推。
- 定义 $d_k(\vec{p}) = |r_k - p_k|$ 表示 C_k 的偏移量（对固定的偏好序列 $\vec{p}$ 而言）。 $D(\vec{p})$ 为长度为 n 的偏好车位序列 $\vec{p}$ 的 $\sum_{i=1}^n d_i(\vec{p})$ 的值，表示序列 $\vec{p}$ 的总偏移量。
- 定义 $t_m(n)$ 为长度等于 n 且满足 $D(\vec{p}) = m$ 的偏好车位序列 $\vec{p}$ 的数量，例如 $t_0(3) = 6$ 。
 $\vec{t}(n) = (t_0(n), t_1(n), t_2(n), \dots)$ 。
- 定义unit-interval CSPF为 $\forall k \in [1, n], d_k(\vec{p}) = |r_k - p_k| \leq 1$ 的CSPF。
- 定义 $D_L(\vec{p})$ 为偏好车位序列 $\vec{p}$ 中车辆从偏好位置开始向左移动的总距离， $D_R(\vec{p})$ 为偏好车位序列 $\vec{p}$ 中车辆从偏好位置开始向右移动的总距离。

- 定义 $\text{Sum}(n) = \sum_{i=0}^{\frac{n(n-1)}{2}} i \cdot t_i(n)$ 表示长度为 n 的 CSPF 总共产生的偏移量的绝对值之和,

$$\text{Sum}(n) = \sum_{\vec{p} \in \text{CSPF}} D(\vec{p}) = \sum_{\vec{p} \in \text{CSPF}} D_L(\vec{p}) + D_R(\vec{p}) = \sum_{\vec{p} \in \text{CSPF}} \sum_{i=1}^n d_i(\vec{p})$$

- 定义 $\delta(\vec{p})$ 表示 $\vec{p}$ 的净偏移量, $\Delta(n)$ 表示长度为 n 的所有 $\vec{p}$ 的净偏移量,

$$\Delta(n) = \sum_{\vec{p} \in \text{CSPF}} D_R(\vec{p}) - D_L(\vec{p}) = \sum_{\vec{p} \in \text{CSPF}} \delta(\vec{p}).$$

下面是围绕 CSPF 展开的一些问题。标记“已解决”的题目代表这道题已经被解决。

- 问题1: 对于第 k 辆车, 它的偏好位置是 p_k , 最终停在 r_k 。 r_k 左边与 α_k 个非空车位相邻, 右边与 β_k 个非空车位相邻。求 p_k 的取值范围。 [已解决](#)
- 问题2: 证明: $t_0(n) = n!$ 。 [已解决](#)
- 问题3: 求 $t_1(n)$ 的表达式。 [已解决](#)
- 问题4: 证明: $t_{\frac{n(n-1)}{2}}(n) = 3$ 。 ($n \in \mathbb{N}, n \geq 3$) [已解决](#)
- 问题5: 求 $t_m(n)$ 的表达式。 ($m \in \mathbb{N}, m \leq \frac{n(n-1)}{2}$)
- 问题6: 探究 unit-interval CSPF。
- 问题7: 探究多项式 $\sum_{\vec{p} \in \text{CSPF}} x^{D_L(\vec{p})} y^{D_R(\vec{p})}$ 。
- 问题8: 广义单峰 (non-strict unimodal) 指的是在一个序列中存在一个点 (或一段连续区间), 使得序列在该点 (或区间) 之前单调不降, 在该点 (或区间) 之后单调不增。探究 $t_m(n)$ 是否是广义单峰的。可能有人在传统停车问题领域研究过这个问题, 需要查资料。
- 问题9: 若一个有限非负实数序列 $a_0, a_1, \dots, a_n$ 被称为对数凹 (log-concave), 则对于所有 $1 \leq i \leq n-1$, 都有 $a_i^2 \geq a_{i-1} \cdot a_{i+1}$ 。探究 $t_m(n)$ 是否对数凹。若 $t_m(n)$ 是对数凹的, 它一定是广义单峰的。可能有人在传统停车问题领域研究过这个问题, 需要查资料。
- 问题10: 尝试证明或推翻 $t_m(n) \leq t_{n-2}(n)$ 。
- 问题11: 求 $t_{n-2}(n)$ 的表达式。
- 问题12: 证明: 当 $n \geq 4$ 时, $t_{\frac{n(n-1)}{2}-1}(n) = 3n$ 。 [已解决](#)
- 问题13: 探究和 $\text{Sum}(n)$ 相关的内容。
- 问题14: 证明 $\Delta(n) = 0$ 。

解答与猜想

这部分是对于 [上述问题](#) 的解答与猜想, 如果问题没有被解决, 那么这里可能会写出一些相关信息。

问题1

p_k 的取值范围

- $p_k \in [p_{k,\min}, p_{k,\max}]$
- $p_{k,\min} = 1$ if $r_k - \alpha_k = 1$, otherwise $p_{k,\min} = r_k - \alpha_k / 2$
- $p_{k,\max} = n$ if $r_k + \beta_k = n$, otherwise $p_{k,\max} = r_k + (\beta_k + 1) / 2$

表达式中涉及的除法均为整数除法。

问题2

$D(\vec{p}) = 0$ 意味着所有车都停到自己的偏好位置。只需要把 n 个车位不重复地分给 n 辆车，一共有 $n!$ 种情况，所以 $t_0(n) = n!$ 。

问题3

$$t_1(n) = \frac{n!(5n-4)}{6} (n \in \mathbb{N}, n \geq 2)$$

用向量 $\vec{q}$ 表示每个车位分别被多少车选作偏好车位。

例如，当 $\vec{p} = (1, 2, 1)$ 时， $\vec{q} = (2, 1, 0)$ ， $q_1 = 2$ 表示 s_1 被两辆车 (C_1 和 C_3) 选作偏好车位， $q_2 = 1$ 表示 s_2 被一辆车 (C_2) 选作偏好车位， $q_3 = 0$ 表示没有车选择 s_3 作为偏好车位。

当 $D(\vec{p}) = 1$ 时，有且仅有一辆车的偏移量为1。因为 C_1 在停车时所有车位都是空车位，所以 C_1 一定能停在偏好车位，即 $d_1(\vec{p}) = 0$ 。因此，偏移量为1的车一定不是 C_1 。

可能满足 $D(\vec{p}) = 1$ 的向量 $\vec{q}$ 有如下两种：

- 第一种
 - $(0, 2, 1, 1, \dots, 1)$
 - $(1, 0, 2, 1, \dots, 1)$
 - ...
 - $(1, 1, \dots, 0, 2, 1)$
 - $(1, 1, \dots, 1, 0, 2)$
- 第二种
 - $(2, 0, 1, 1, \dots, 1)$
 - $(1, 2, 0, 1, \dots, 1)$
 - ...
 - $(1, 1, \dots, 2, 0, 1)$
 - $(1, 1, \dots, 1, 2, 0)$

两种向量 $\vec{q}$ 都有 $n - 1$ 种排列方式。加起来一共是 $2(n - 1)$ 种。

将 n 个车位按照这些向量 $\vec{q}$ 分给 n 辆车，其中值为 2 的 $\vec{q}$ 的项对应的两辆车不需要排序，得到这些向量 $\vec{q}$ 对应 $\frac{n!}{2!} \cdot 2(n - 1)$ 种 $\vec{p}$ ，也就是 $n!(n - 1)$ 。

但是这些向量 $\vec{q}$ 不全满足 $D(\vec{p}) = 1$ 。比如 $\vec{p} = (1, 5, 3, 3, 2)$ 对应 $\vec{q} = (1, 1, 2, 0, 1)$ ，但是 $D(\vec{p}) = d_4(\vec{p}) + d_5(\vec{p}) = 1 + 2 = 3 \neq 1$ 。所以还需要从 $n!(n - 1)$ 中减去一部分，命名这部分为 b ，所以 $t_1(n) = n!(n - 1) - b$ 。

第一种 $\vec{q}$ 中有两个选择了同一个车位座位偏好车位的车，其中后来的车在发现车位已经被占的时候移动到该车位左侧的那个车位（无论它的偏好车位右侧的车位是否有车），偏移量为1，其他的车不会发生偏移，所以第一种 $\vec{q}$ 一定满足 $D(\vec{p}) = 1$ 。因此， b 完全来自第二种 $\vec{q}$ 。

对于格式符合 $(2, 0, 1, 1, \dots, 1)$ 的 $\vec{q}$ ，选择了 s_1 的两辆车中的一辆一定会偏移到 s_2 ，其他车的偏移量均为0，符合 $D(\vec{p}) = 1$ 。因此， $\vec{q} = (2, 0, 1, 1, \dots, 1)$ 是符合 $D(\vec{p}) = 1$ 的，所以 b 完全来自剩余的第三种 $\vec{q}$ 。

设 $\vec{q} = (1 \dots, 2, 0 \dots)$ （省略的项都是1，而且省略的部分可能没有项），且 $q_i = 2, q_{i+1} = 0$ 。

若 $p_x = i, p_y = i, p_z = i - 1, x < y, D(\vec{p}) = 1$ ，则 $d_y(\vec{p}) = 1$ 。在这种情况下， C_y 向右移动了一个单位，这意味着 s_{i-1} 已经有车了。因为 $D(\vec{p}) = d_y(\vec{p}) = 1$ ，所以 $d_z(\vec{p}) = 0$ ，所以 C_z 停在 s_{i-1} ，而且 C_z 在 C_y 之前停车（ $y > z$ ）。那么，当 $x < y < z$ 时， $D(\vec{p}) \neq 1$ 。

b 对应 $n - 1 - 1 = n - 2$ 种 $\vec{q}$ （从 $n - 1$ 个第二种 $\vec{q}$ 中删去一种），从 n 辆车里选出三辆，让它们中序号最大的那一辆车的偏好位车位是 s_{i-1} ，剩下的两辆的偏好车位是 s_i ，然后把除 s_{i-1}, s_i 和 s_{i+1} 外的 $n - 3$ 个车位分配给剩下的 $n - 3$ 辆车，得到 $b = \binom{n}{3}(n - 2)(n - 3)!$ 。所以

$$t_1(n) = n!(n - 1) - b = n!(n - 1) - \binom{n}{3}(n - 2)(n - 3)! = \frac{n!(5n - 4)}{6}。$$

问题4

如果在序列 $\vec{p}$ 中，所有车的偏移量都达到它自己的最大值，那么 $D(\vec{p})$ 一定达到它的最大值。

对于第 i 辆车，理论上它的最大偏移量是 $i - 1$ ，即经过所有已经停车的车。以 C_3 为例，要想让 $d_3(\vec{p}) = 2$ ，它必须经过 C_1 和 C_2 。如果 C_1 和 C_2 不连续，则 $d_3(\vec{p}) \leq 1$ ，所以 C_1 和 C_2 连续。如果 $s_1 = s_n = 0$ ，那么 $d_3(\vec{p}) \leq 1$ 。

更普遍地，对于长度为 n 的偏好车位序列 $\vec{p}$ 和 C_i （ $i \in \mathbb{N}, 3 \leq i \leq n$ ），如果存在 $x, y \in \mathbb{N}, x < y \leq n$ 满足 $s_{x+1} = 0$ 或 $s_{y-1} = 0$ ，那么 $d_i(\vec{p}) \leq i - 2 < i - 1$ 。如果 $s_1 = s_n = 0$ ，那么 $d_i(\vec{p}) \leq i/2 \leq i - 1$ ，当且仅当 $i = 2$ 时， $i/2 = i - 1$ 。

所以，对于 $i \in \mathbb{N}, 3 \leq i \leq n$ ，要让 $d_i(\vec{p}) = i - 1$ ，必须让已经停车的车停在序号连续的 $i - 1$ 个车位，并且 $s_1 \neq 0$ 或 $s_n \neq 0$ 。若 C_i 准备停车，已经停车的车停在序号连续的 $i - 1$ 个车位，则 $s_1 \neq 0$ 和 $s_n \neq 0$ 不会同时发生。如果 C_i 准备停车，已经停车的车停在序号连续的 $i - 1$ 个车位，且 $s_1 \neq 0, s_n \neq 0$ ，那么这意味着所有车位都有车，但至少还有一辆车没有停车，且车位数量等于车数量，所以至少应该有一个空车位，矛盾。所以当 C_i 准备停车，且已经停车的车停在序号连续的 $i - 1$ 个车位时， $s_1 \neq 0$ 和 $s_n \neq 0$ 不会同时发生。而要让 $D(\vec{p})$ 达到最大值时， s_1 和 s_n 不能同时等于0，所以 s_1 或 s_n 为0。

下面在已经停车的 $i - 1$ 辆车停在连续的 $i - 1$ 个车位的条件下分情况讨论：

- $s_1 \neq 0$
 - 在这种情况下，只有 $i \in \mathbb{N}, 3 \leq i \leq n, p_i = 1$ ，才能确保 C_i 经过所有 $i - 1$ 辆车。 $d_1(\vec{p})$ 恒等于0，因为 C_1 在停车时偏好车位不会被任何车占用。 $d_{2,\max}(\vec{p}) = 1, d_2(\vec{p}) = 1$ 有两种情况，分别是 $s_1 = s_2 = 1$ 和 $s_1 = s_2 \geq 2$ 。当 $s_1 = s_2 = 1$ 时， C_2 会从 s_1 移动到 s_2 。当 $s_1 = s_2 \geq 2$ 时， C_2 会从 p_2 移动到 p_2 左侧相邻的车位。为了确保 C_3 能经过 C_1 和 C_2 而且 $p_3 = 1$ ，理论上有且仅有两种情况，即 $s_1 = 1, s_2 = 2$ 和 $s_1 = 2, s_2 = 1$ 。两种情况分别对应

$p_1 = p_2 = 1$ 和 $p_1 = p_2 = 2$, 结合当 $i \in \mathbb{N}, 3 \leq i \leq n$ 时, $p_i = 1$ 得到 $s_1 \neq 0$ 时能让 $D(\vec{p})$ 达到最大值的 $\vec{p}$ 可以是 $(1, 1, 1, \dots)$ 或 $(2, 2, 1, 1, 1, \dots)$ 。

- $s_n \neq 0$
 - 在这种情况下, 只有 $i \in \mathbb{N}, 3 \leq i \leq n, p_i = n$, 才能确保 C_i 经过所有 $i - 1$ 辆车。
 $d_1(\vec{p}) = 0, d_2(\vec{p}) \leq 1$.当且仅当 $s_1 = s_2 = n$ 时能够使 C_i 经过所有 $i - 1$ 辆车。所以当 $s_n \neq 0$ 时, 让 $D(\vec{p})$ 达到最大值的 $\vec{p}$ 是 $(n, n, n, \dots)$ 。

当 $n \geq 3$ 时, $D(\vec{p})$ 的最大值是 $\sum_{i=1}^n i - 1 = \frac{n(n-1)}{2}$, 所以 $t_{\frac{n(n-1)}{2}} = 3$, 这三种 $\vec{p}$ 分别是 $(1, 1, 1, \dots)$, $(2, 2, 1, 1, 1, \dots)$ 和 $(n, n, n, \dots)$ 。

问题5

一味地从数列推导, 或者列出各种可能的情况或许是无效的。我有了新的思路:

最终的结果很有可能是 $t_m(n) = f(m, n) - g(m, n) - h(m, n)$ 的格式,

$f(m, n) = f_1(m, n) + f_2(m, n)$, $f_1(m, n)$ 代表从 $t_{m-1}(n)$ 种情况中延伸出的能让 $q_i = 0$ 的车位数量不变的情况数量, $f_2(m, n)$ 代表从 $t_{m-1}(n)$ 种情况中延伸出的能让 $q_i = 0$ 的车位数量加1的情况数量, $g(m, n)$ 代表 $f_1(m, n)$ 和 $f_2(m, n)$ 重合的情况, $h(m, n)$ 代表 $f(m, n)$ 中可能存在的不符合要求的情况。

我大胆猜测, 对于所有的 n , $\frac{\partial g}{\partial m} \geq 0$ 。如果 n 足够大, $\frac{\partial f}{\partial m} \geq 0$ 。

问题6

unit-interval CSPF 数量

[1, 4, 20, 135, 1136, 11488, 135547]

问题7

问题8

和问题5有关。

问题9

和问题5有关。

问题10

和问题5有关。猜想可能是错误的。从 $n \geq 10$ 开始, $t_m(n)$ 的最大值等于 $t_{n-1}(n)$, 和之前的猜测不同。但是 $n \geq 10$ 的代码是AI生成的, 可能导致结果错误。

问题11

和问题5有关。尚未确定是否需要解决。问题10的猜想可能错误。

问题12

当 $n \geq 4$ 时, $D_{\max}(\vec{p}) = \frac{n(n-1)}{2}$, 所以当且仅当这 n 辆车中有一辆车 C_i ($i \neq 1$) 满足

$d_i(\vec{p}) = d_{i,\max}(\vec{p}) - 1 = i - 2$, 且其他车 C_j ($i \neq j$) 满足 $d_j(\vec{p}) = d_{j,\max}(\vec{p}) = j - 1$ 时,

$$D(\vec{p}) = \frac{n(n-1)}{2} - 1。$$

也就是说, 若要让 $D(\vec{p}) = \frac{n(n-1)}{2} - 1$, 只需要在三种满足 $D(\vec{p}) = D_{\max}(\vec{p})$ 的 $\vec{p}$ 的基础上, 各修改一辆车的偏好车位即可。

讨论三种满足 $D(\vec{p}) = D_{\max}(\vec{p})$ 的 $\vec{p}$:

- $\vec{p} = (1, 1, 1, 1, \dots)$
 - 在这种情况下, $r_k = k$ 。新进来的车 C_k 从 s_1 开始经过所有已经停进去的 $k-1$ 辆车, 到达第一个空车位 s_k (这也是最近的空车位)。以 C_4 为例, 在这种情况下, 当且仅当 $p_4 = 2$ 时, $d_4(\vec{p}) = d_{4,\max}(\vec{p}) - 1$ 。 C_i ($i \geq 4$)同理, 当且仅当 $p_i = 2$ 且 $p_j = 1$ ($i \neq j$)时, $d_i(\vec{p}) = d_{i,\max}(\vec{p}) - 1$, $D(\vec{p}) = \frac{n(n-1)}{2} - 1$ 。令 $\vec{p}_1$ 满足 $p_i = 2$ ($i \neq 1$), 且对于所有 $j \neq i$, $p_j = 1$, 这样的 $\vec{p}_1$ 有 $n-1$ 个。
- $\vec{p} = (n, n, n, n, \dots)$
 - 同理, 若 $i \geq 4$, 则当且仅当 $p_i = n-1$ 且 $p_j = n$ ($i \neq j$)时, $d_i(\vec{p}) = d_{i,\max}(\vec{p}) - 1$, $D(\vec{p}) = \frac{n(n-1)}{2} - 1$ 。令 $\vec{p}_2$ 满足 $p_i = n-1$ ($i \neq 1$), 且对于所有 $j \neq i$, $p_j = n$, 这样的 $\vec{p}_2$ 有 $n-1$ 个。
- $\vec{p} = (2, 2, 1, 1, 1, \dots)$
 - 同理, 若 $i \geq 4$, 则当且仅当 $p_1 = p_2 = p_i = 2$ 且 $p_j = 1$ ($i \neq j, j \geq 3$)时, $d_i(\vec{p}) = d_{i,\max}(\vec{p}) - 1$, $D(\vec{p}) = \frac{n(n-1)}{2} - 1$ 。令 $\vec{p}_3$ 满足 $p_1 = p_2 = p_i = 2$ ($i \geq 3$), 且对于所有 $j \neq i$, $p_j = 1$, 或 $\vec{p}_3 = (2, 1, 1, 1, 1, \dots)$, 这样的 $\vec{p}_3$ 有 $n-1$ 个。

注意, 若要让 $d_4(\vec{p}) \geq 2$, 则前三辆车必须停在相邻的三个车位, 且有且仅有一辆车停在 s_1 或 s_n 。 $d_k(\vec{p})$ ($k \geq 4$)同理。

所以, 当 $i \geq 4$ 时, 满足 $d_i(\vec{p}) = d_{i,\max}(\vec{p}) - 1$ 的 $\vec{p}$ 有3个。

若 $d_i(\vec{p}) = d_{i,\max}(\vec{p}) - 1$, 则 $i \neq 1$, 因为 C_1 的偏好车位不可能被占用。

如果用同样的办法看向 C_2 和 C_3 , 那么 $t_{\frac{n(n-1)}{2}-1}(n) = 3(n-1)$, 和观测得到的结果不同。

首先看向 C_3 , 按照之前的思路, 若 $d_3(\vec{p}) = i - 2 = 1$, 应该有如下情况:

- $\vec{p}_1 = (1, 1, 2, 1, \dots)$
- $\vec{p}_2 = (n, n, n-1, n, \dots)$
- $\vec{p}_3 = (2, 2, 2, 1, 1, 1, \dots)$

除了这三种情况外, 还有两种特殊情况满足 $d_3(\vec{p}) = i - 2 = 1$:

- $\vec{p}_4 = (3, 3, 2, 1, 1, 1, 1, \dots)$
- $\vec{p}_5 = (n-1, n-1, n-1, n, n, n, n, \dots)$

然后看向 C_2 ，有一种特殊情况满足 $d_2(\vec{p}) = i - 2 = 0$ ：

- $\vec{p}_6 = (n-1, n, n, n, n, \dots)$

所以，当 $n \geq 4$ 时， $t_{\frac{n(n-1)}{2}-1}(n) = 3(n-1) + 1 + 2 = 3n$ 。

当 $n = 3$ 时，若 $D(\vec{p}) = D_{\max}(\vec{p}) - 1$ ，则 $\vec{p}_2 = (3, 3, 2)$ ， $\vec{p}_3 = (2, 2, 2)$ ， $\vec{p}_4 = (3, 3, 2)$ ， $\vec{p}_5 = (2, 2, 2)$ ，所以 $\vec{p}_2 = \vec{p}_4$ ， $\vec{p}_3 = \vec{p}_5$ 。

这意味着 $n = 3$ 时有两种特殊情况因为和其他情况恰好相同而被合并。所以当 $n = 3$ 时，

$t_{\frac{n(n-1)}{2}-1}(n) = 3(n-1) + 1 = 3n - 2$ 。

当 $n = 2$ 时，若 $D(\vec{p}) = D_{\max}(\vec{p}) - 1$ ，则 $\vec{p}_4$ 因为索引溢出而不存在。 $\vec{p}_5$ 因为需要三辆车的偏好车位是 $n-1$ ，而 $n = 2 < 3$ ，所以 $\vec{p}_5$ 也不存在。 $\vec{p}_6 = (1, 2)$ ， $\vec{p}_1 = (1, 2)$ ，所以 $n = 2$ 时有两种特殊情况不存在，一种特殊情况因为和其他情况恰好相同而被合并。此外， $\vec{p}_3$ 需要 $n \geq 3$ 才能实现，所以 $\vec{p}_3$ 对应的 $n-1$ 个情况不存在。因此，当 $n = 2$ 时， $t_{\frac{n(n-1)}{2}-1}(n) = 3(n-1) - (n-1) = 2n - 2$ 。

因为当 $n = 1$ 时， $\frac{n(n-1)}{2} - 1 = -1$ ，而根据定义， $D_{\max}(\vec{p}) = \sum_{i=1}^n |r_i - p_i| \geq 0$ ，矛盾，所以当

$n = 1$ 时， $t_{\frac{n(n-1)}{2}-1}$ 不存在。

综上所述，

$$t_{\frac{n(n-1)}{2}-1}(n) = \begin{cases} 2(n-1) = 2n - 2 & \text{if } n = 2, \\ 3(n-1) + 1 = 3n - 2 & \text{if } n = 3, \\ 3(n-1) + 1 + 2 = 3n & \text{if } n \geq 4 \end{cases}$$

问题13

Sum(n)似乎是个偶数

问题14

数据

这里有一些重要数据。

$t_m(n)$ 序列

```
n=1: [1]
sum = 0
```

```
n=2: [2, 2]
sum = 2
```

n=3: [6, 11, 7, 3]

sum = 34

n=4: [24, 64, 70, 52, 31, 12, 3]

sum = 562

n=5: [120, 420, 637, 637, 526, 378, 229, 112, 48, 15, 3]

sum = 10306

n=6: [720, 3120, 6044, 7592, 7604, 6705, 5388, 3920, 2544, 1511, 832, 411, 178, 66, 18, 3]

sum = 211776

n=7: [5040, 26040, 61362, 93097, 109245, 110787, 102599, 88573, 71531, 53841, 37998, 25574, 16494, 10053, 5708, 3041, 1508, 676, 265, 87, 21, 3]

sum = 4869356

n=8: [40320, 241920, 671328, 1197120, 1620394, 1851530, 1905920, 1828004, 1658043, 1430855, 1175434, 921087, 694677, 508867, 362710, 250341, 166268, 106145, 65241, 38449, 21506, 11330, 5601, 2560, 1052, 376, 111, 24, 3]

sum = 124217536

n=9: [362880, 2479680, 7917840, 16249656, 25025013, 31939007, 36090435, 37635471, 37067260, 34878807, 31529945, 27446686, 23049733, 18757971, 14887647, 11584136, 8849786, 6621676, 4836720, 3444507, 2393020, 1622653, 1072283, 687801, 426548, 255268, 147114, 81201, 42587, 21057, 9727, 4126, 1566, 514, 138, 27, 3]

sum = 3490099468

n=10: [3628800, 27820800, 100457280, 233397120, 404070588, 572202448, 706720634, 795256853, 838717290, 843223551, 815655100, 762901636, 691945121, 609781764, 523491582, 439568309, 362637839, 294920443, 236740766, 187411521, 146023777, 111838346, 84203724, 62356355, 45417994, 32506466, 22822564, 15690995, 10552637, 6937684, 4453458, 2784810, 1691089, 994334, 564540, 308300, 160973, 79745, 37158, 16101, 6374, 2248, 682, 168, 30, 3]

sum = 107171657344

n=11: [39916800, 339292800, 1366817760, 3547967280, 6832956174, 10664174901, 14343954001, 17369660460, 19521117840, 20783950132, 21239440165, 20993564892, 20159256217, 18858736654, 17221049549, 15380497452, 13470662650, 11606716696, 9869488429, 8301735076, 6914999429, 5702305863, 4650518570, 3747424947, 2982707910, 2345568293, 1822979788, 1400200611, 1062315483, 795442565, 587310678, 427308732, 306235566, 216088775, 150026072, 102372463, 68564094, 45013840, 28935858, 18189157, 11160622, 6668152, 3868364, 2172303, 1176619, 611812, 303482, 142509, 62764, 25606,


```
9505, 3131, 883, 201, 33, 3]  
sum = 3573968348686
```

其中 $n \geq 10$ 的序列由AI生成的代码运行得到。更大的 n 对应的序列正在计算，后续可能补充。